E-Commerce Analytics Platform — Technical Report

Course: Advanced Application Development (CSE 214)

Project: E-Commerce Analytics Platform with Multi-Agent Text2SQL AI Chatbot

Yaren Erişen 20240808601

Elif Polatdemir 20230808612

GitHub: https://github.com/elifpdemir/ecommerce

Table of Contents

1. Executive Summary
2. System Architecture and Design Decisions
3. Database Design and ER Diagram
4. ETL Process and Field Mappings
5. API Documentation Summary
6. AI Chatbot Architecture (Text2SQL)
7. Security Architecture
8. Challenges Encountered and Solutions
9. Conclusion and Future Improvements

1. Executive Summary

This project aims to develop an e-commerce analytics platform inspired by modern business intelligence tools (similar to Metabase), supporting three different user roles (Admin, Corporate, Individual). The platform operates on a normalized relational database obtained by integrating six different Kaggle datasets and is enhanced with a multi-agent artificial intelligence chatbot capable of generating SQL from natural language.

Technology Stack (Implementation):

- Backend: Spring Boot 3.2.5 (Java 21), Spring Security, Spring Data JPA, JJWT 0.11.5
- Frontend: Angular (ecommerce/frontend)
- Database: MySQL / MariaDB 10.4.28 (ecommerce_analytics)
- AI Integration: Text2SQL engine based on Google Gemini 2.5 Flash (Generative Language API)
- Build: Maven; AI Integration Path: REST (RestTemplate)

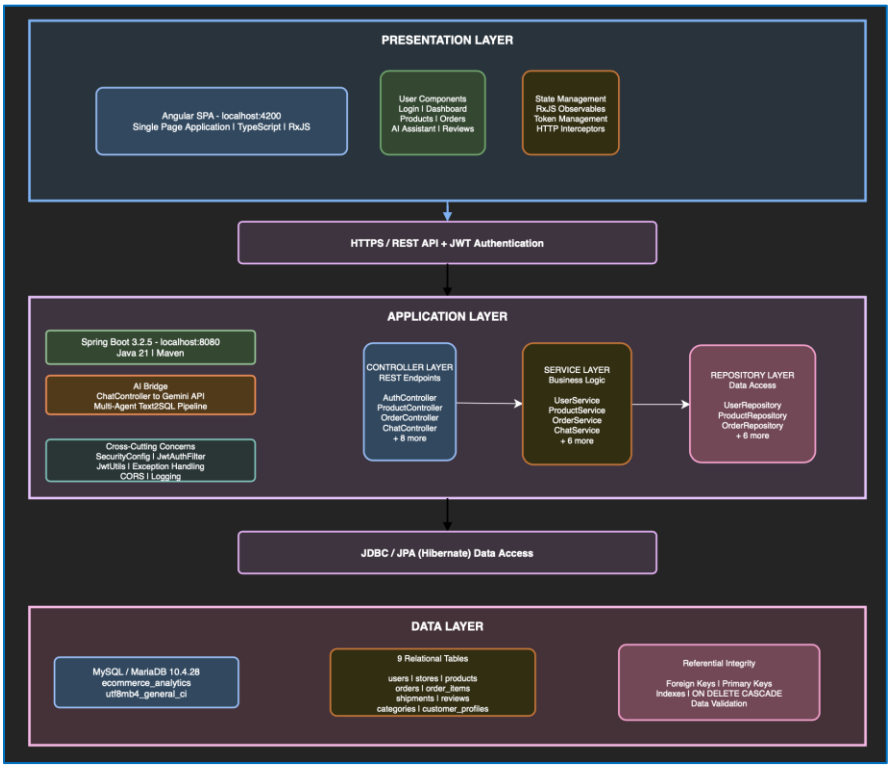
Key Capabilities:

- Multi-tenant design with role-based access control (RBAC)
- JWT-based authentication and infrastructure for refresh token support
- Full CRUD services over 9 core entities
- Natural language query → SQL transformation → secure execution → formatted response chain
- Automatic blocking of sensitive fields (such as password and hash) in AI flows (guardrails)

2. System Architecture and Design Decisions

2.1 High-Level Architecture

The system is designed with a 3-tier architecture, and each layer can scale independently:



2.2 Architectural Decisions and Rationales

Decision	Selected Solution	Rationale
Backend Framework	Spring Boot 3.2.5	Mature ecosystem, RBAC integration with Spring Security, multi-database support with JPA
Java Version	Java 21 (LTS)	Virtual threads, modern language features, and long-term support
Authentication	Stateless JWT	Friendly for horizontal scaling, no server-side session; ideal for SPA
Data Layer	JPA/Hibernate + JdbcTemplate	JPA for CRUD, JdbcTemplate for dynamic (chatbot) queries
Database	MySQL/MariaDB	phpMyAdmin management, broad hosting support; portable to PostgreSQL via application.properties
AI Provider	Google Gemini 2.5 Flash	Low latency, free tier, JSON-based generative text API
Frontend	Angular	Modular component architecture, asynchronous data flow with RxJS, mature for enterprise projects
API Style	REST + JSON	Client-independent, can be documented with OpenAPI

2.3 Module Structure

Under the package com.example.ecommerce_project, a standard Spring Boot layered package structure has been applied:

Layer	Content Summary	Description
Config	SecurityConfig	Handles security and CORS configurations.
Security	JwtUtils, JwtAuthFilter	Implements JWT-based authentication and authorization.
Entity	Database Models	Core database objects (User, Product, Order, etc.).
Repository	Data Access Layer	Manages database CRUD operations via Spring Data JPA.
Service	Business Logic	Contains core business rules and ChatService operations.
Controller	API Endpoints	RestController classes that handle HTTP requests.
DTO	Data Transfer Objects	Models used for data transfer between application layers.

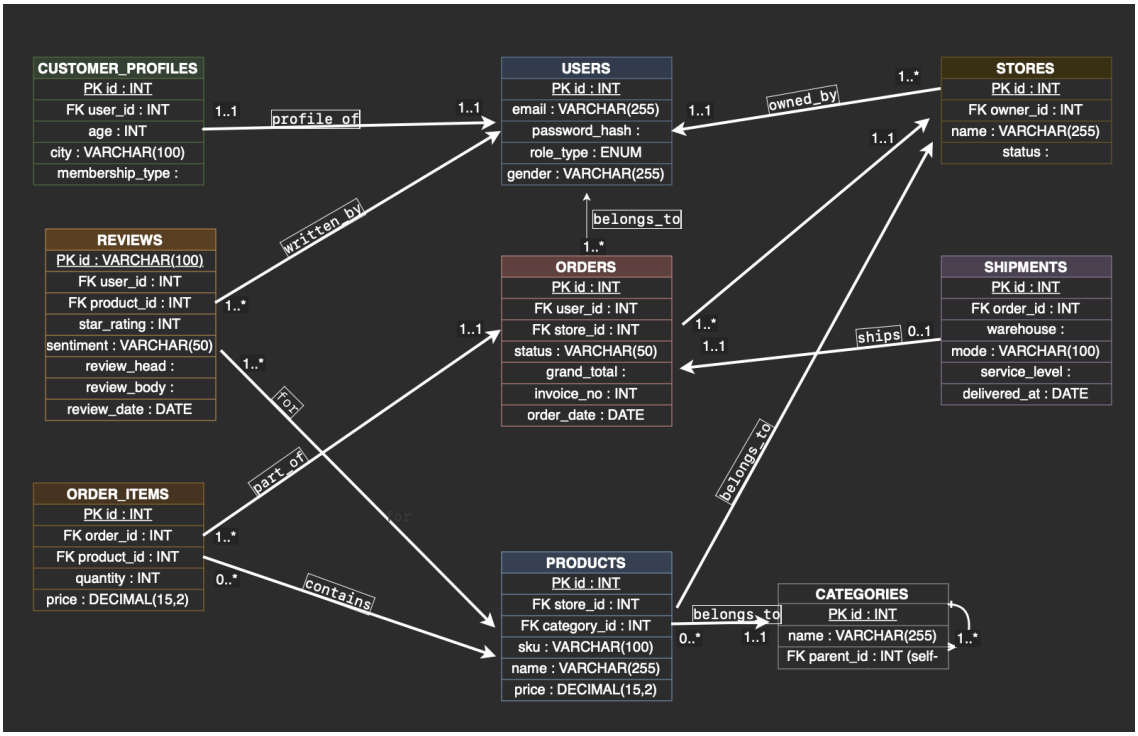
This layered structure ensures unidirectional dependencies (Controller → Service → Repository) and enables independent testing of each layer.

3. Database Design and ER Diagram

3.1 Schema Overview

The database contains 9 tables normalized to 3NF (third normal form) level. All foreign keys are defined with ON DELETE behaviors, and indexes are added to frequently used fields.

3.2 ER Diagram



3.3 Table Details

USERS (users)

Column	Type	Constraint
id	INT	PK, AUTO_INCREMENT
email	VARCHAR(255)	NOT NULL, UNIQUE INDEX (idx_user_email)
password_hash	VARCHAR(255)	NOT NULL
role_type	ENUM('ADMIN', 'CORPORATE', 'INDIVIDUAL')	NOT NULL
gender	VARCHAR(255)	NULL

STORES (stores) — Stores linked to corporate users

• PK id; FK owner_id → users.id; status ENUM('ACTIVE','PASSIVE') PRODUCTS (products) — Store catalog items

• PK id; FK store_id → stores.id; FK category_id → categories.id; sku VARCHAR(100), price DECIMAL(15,2)

ORDERS (orders) — Order header

• PK id; FK user_id → users.id; FK store_id → stores.id; status, grand_total DECIMAL(20,2), invoice_no, order_date DATE

ORDER_ITEMS (order_items) — Order lines
• PK id; FK order_id → orders.id; FK product_id → products.id; quantity, price

SHIPMENTS (shipments) — Shipping information
• PK id; FK order_id → orders.id; warehouse, mode, service_level, delivered_at

REVIEWS (reviews) — Review and rating

• PK id VARCHAR(100) (Amazon original review_id); FK user_id, FK product_id; star_rating, sentiment, review_head, review_body, review_date

CATEGORIES (categories) — Hierarchical categories

• PK id; FK parent_id → categories.id (self-referencing) — tree structure with the adjacency list model

CUSTOMER_PROFILES (customer_profiles) — Individual user details

• PK id; FK user_id → users.id (CASCADE DELETE); age, city, membership_type (Diamond/Gold/Silver/Bronze)

3.4 Indexes and Referential Integrity

- PRIMARY KEY exists in all tables.
- idx_user_email index on users.email (for login queries).
- 9 foreign key constraints; ON DELETE SET NULL ON UPDATE CASCADE on categories.parent_id, ON DELETE CASCADE on customer_profiles.user_id.
- ENGINE=InnoDB + utf8mb4_general_ci — full support for Turkish and international character sets.

4. ETL Process and Field Mappings

Six different Kaggle datasets were reduced into a single relational schema through integration and normalization processes.

4.1 Data Integration Matrix

Target Table	DS1 Online Retail	DS2 Customer Behavior	DS3 Shipping	DS4 Amazon Sales	DS5 Pakistan E-Comm	DS6 Reviews
users	CustomerID	Demographics	Gender	—	—	CustomerID
products	StockCode, Description	—	CostOfProduct	SKU, Style	SKU	ProductTitle
orders	InvoiceNo, InvoiceDate	—	—	Status	Full Order, Payment	—
order_items	Quantity, UnitPrice	—	—	—	QtyOrdered, Price	—
shipments	—	—	All fields	ShipServiceLevel	—	—
reviews	—	AvgRating	—	—	—	StarRating, Body
customer_profiles	—	Age, City, Membership, TotalSpend	—	—	—	—
categories	—	—	—	Category	CategoryName	ProductCategory

4.2 Field Mapping Document

USERS Table

Target Field	Source	Transformation
id	Generated surrogate key	AUTO_INCREMENT
email	Derived contact field	LowerCase + trim
password_hash	Synthetic	Fixed hash for test data
role_type	Inferred	MembershipType → ADMIN/CORPORATE/INDIVIDUAL mapping
gender	DS2, DS3	M/F/Other normalization

PRODUCTS Table

Target Field	Source	Transformation
id	DS5, DS1 product refs	Integer allocation (211131+)
sku	DS1, DS5, DS4	Combined; format BTM038-PP-XXXL
name	DS1, DS6	Trim + shortening (255 char limit)
price	DS1, DS5	Converted to USD; DECIMAL(15,2)
category_id	DS4, DS5, DS6	Mapped through category dictionary
store_id	Assigned	Round-robin assignment

ORDERS Table

Target Field	Source	Transformation
id	DS5, DS1	Surrogate reduction into one id field
invoice_no	DS1	Original number preserved
status	DS4, DS5	Unified labels (e.g., "Shipped", "Cancelled")
grand_total	DS5, DS1	SUM(qty * unit_price)
order_date	DS1, DS5	ISO 8601 (YYYY-MM-DD)
user_id, store_id	Association mapping	Foreign key resolution

ORDER_ITEMS Table

Target Field	Source	Transformation
quantity	DS1, DS5	Negative returns filtered out
price	DS1, DS5	Instant unit price per row

SHIPMENTS Table

Target Field	Source	Transformation
warehouse	DS3	Single-letter code (A/B/C/D/F)
mode	DS3	Flight / Ship / Road
service_level	DS4	Standard / Expedited
delivered_at	Calculated from DS3	Order + transit time (NULL if undelivered)

REVIEWS Table

Target Field	Source	Transformation
id	DS6	Original R... format preserved (string PK)
star_rating	DS6, DS2	1-5 scale
sentiment	Inferred from DS6 text	"Satisfied", "Neutral", "Dissatisfied"
review_head	DS6	200 character trim
review_body	DS6	800 character trim; HTML cleanup
review_date	DS6	ISO 8601

CUSTOMER_PROFILES Table

Target Field	Source	Transformation
age	DS2	NULL if missing
city	DS2	Initial capitals (Title Case)
membership_type	DS2	Diamond / Gold / Silver / Bronze

CATEGORIES Table

- A flat list was converted into a hierarchical tree:
- 4 top categories (90: Digital Content and Software, 91: Physical Media, 92: Electronics, 93: Lifestyle)
- 14 subcategories linked by parent_id.

4.3 Data Cleaning Steps

1. 1. Missing Value Handling: Rows were dropped if required fields (email, sku, order_date, etc.) were missing; optional fields were written as NULL.
2. 2. Duplicate Removal: Duplicate records were eliminated with DISTINCT on email and sku.
3. 3. Format Standardization:

- Dates: All dates were converted to ISO 8601 (YYYY-MM-DD) format.
- Currency: All prices were converted to a single currency (USD) and DECIMAL(15,2) precision; DECIMAL(20,2) for grand_total (for large totals).
- Encoding: All tables support Turkish and international characters with utf8mb4_general_ci.

4. 4. Surrogate Key Generation: All surrogate keys such as users.id, orders.id, and products.id were generated by MySQL with AUTO_INCREMENT. reviews.id preserved Amazon's original value.
5. 5. Reference Resolution: Values such as customer_id, order_id, and product_id in integrated datasets were mapped to new surrogate keys to ensure foreign key integrity.

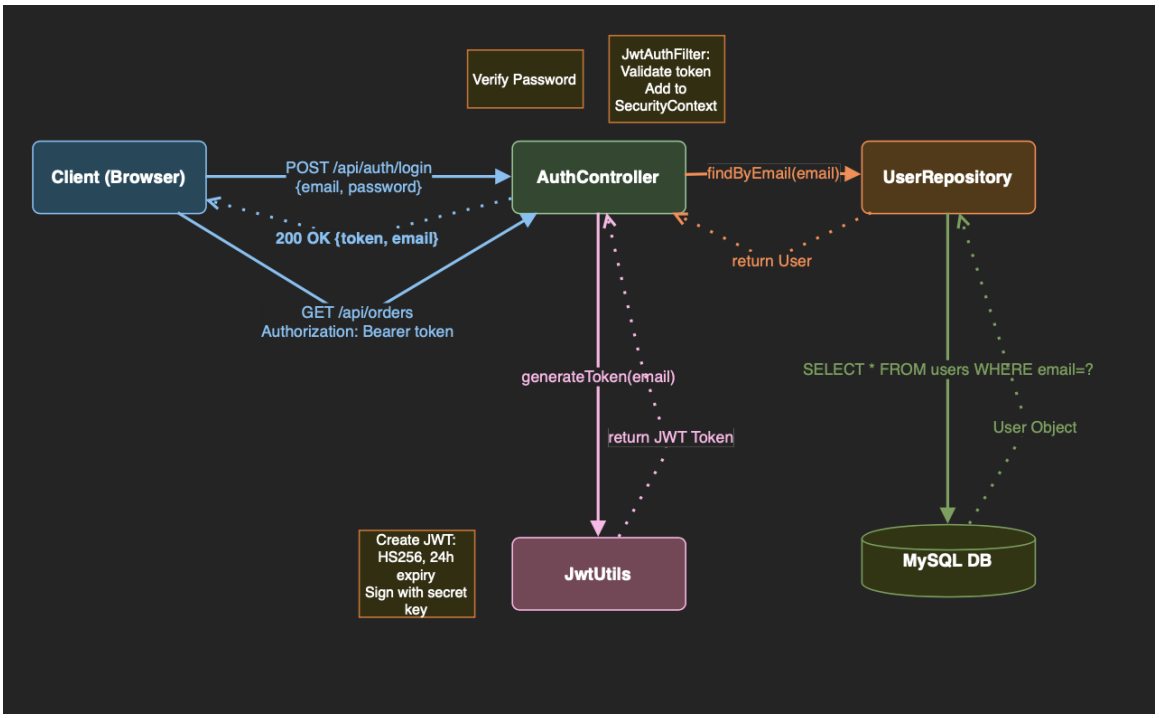
5. API Documentation Summary

The API consists of JSON-carrying, JWT-protected endpoints compatible with REST architecture. All controllers are accessible from the Angular client under /api/... or resource names via CORS (http://localhost:4200)

5.1 Endpoint Inventory

Controller	HTTP Path	Method	Description	Access
AuthController	/api/auth/login	POST	Email + password; 200 → JWT, 401 → rejected	Public
UserController	/api/users	GET/POST/PUT/DELETE	User CRUD	ADMIN
StoreController	/api/stores	GET/POST/PUT/DELETE	Store management	ADMIN, CORPORATE
ProductController	/products	GET	All products; ?bestseller=truereturns top 3	Public Read
ProductController	/products/categories	GET	All category names	Public
CategoryController	/api/categories	GET/POST	Category tree	Mixed
OrderController	/api/orders	GET	All orders	Auth Required
OrderItemController	/api/order-items	GET	Order lines	Auth Required
ShipmentController	/api/shipments	GET	Shipment status	Mixed
ReviewController	/api/reviews	GET/POST	Product reviews	INDIVIDUAL can post
CustomerProfileController	/api/customer-profiles	GET	User profile	Auth Required
ChatController	/api/chat/ask	GET	Natural language query → JSON response	All Roles

5.2 Authentication Flow



JWT Details (JwtUtils.java):

- Algorithm: HS256
- Validity: 24 hours (86,400,000 ms)
- Secret: read from app.jwt.secret property
- Claim: subject = email

5.3 Error Handling

- 401: Invalid password
- 404 / RuntimeException: "User Not Found"
- 500: General server error
- AI endpoint specifically returns human-readable messages (" Access to this data was blocked due to security protocol...")

5.4 Request and Response Examples

Login Request:

```
POST /api/auth/login
Content-Type: application/json
{ "email": "user@example.com", "password": "secret123" }
```

Login Response (200 OK):

```
{ "token": "eyJhbGciOiJIUzI1NiJ9...", "email": "user@example.com" }
```

Chat Request:

```
GET /api/chat/ask?message=What are the top 5 best-selling products?
```

Chat Response: Formatted analysis report in Markdown format (see § 6).

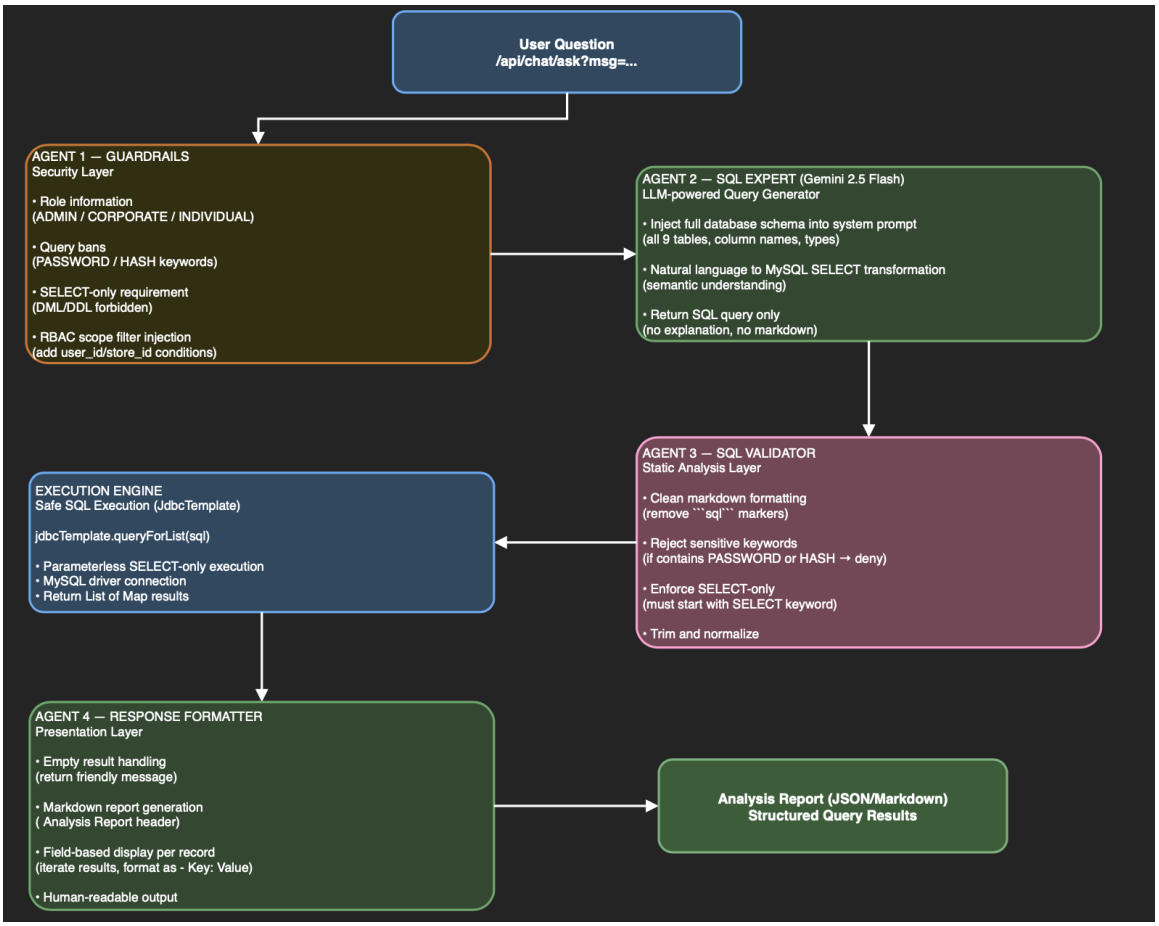
6. AI Chatbot Architecture (Text2SQL)

6.1 Problem Definition

In a classic e-commerce platform, the sales and marketing team cannot be expected to know SQL; however, they need the fastest insights. The AI Assistant in this platform automatically converts natural language questions into SQL, executes them securely, and returns results as a readable analysis report.

6.2 Multi-Agent Approach — Conceptual Flow

The project applies a logically multi-agent approach inside a single orchestrator service (ChatService). Each "agent" conceptually has a separate responsibility:



6.3 Agent Responsibilities

Agent	Role	Responsibility
Guardrails Agent	Security and Scope Manager	Enriches question with security rules; injects role-based restrictions.
SQL Expert Agent	Gemini 2.5 Flash	Generates schema-aware SQL queries.
SQL Validator	Static Analysis	Validates against forbidden keywords and enforces SELECT-only requirements.
Execution Engine	JdbcTemplate	Executes SQL securely and returns a list of result rows
Response Formatter	Data Analyst	Converts raw results into a human-friendly Markdown summary.

6.4 System Prompt (Schema Injection)

Analysis Report

Record 1

- **Product Name:** Wireless Headphones
- **Total Sales:** 1,284
- **Average Rating:** 4.6

You are an e-commerce data analyst. Return only valid MySQL queries.
Our tables:
1. users (id, email, role_type, gender)
2. stores (id, owner_id, name, status)
3. shipments (id, order_id, warehouse, mode, service_level, delivered_at)
4. reviews (id, user_id, product_id, star_rating, sentiment, ...)
5. products (id, store_id, category_id, sku, name, price)
6. order_items (id, order_id, product_id, quantity, price)
7. orders (id, user_id, store_id, status, grand_total, invoice_no, order_date)
8. customer_profiles (id, user_id, age, city, membership_type)
9. categories (id, name, parent_id)

SECURITY RULES:
- Current user: <email>, Role: <ADMIN|CORPORATE|INDIVIDUAL>
- Never return data containing passwords or hashes.
- If role is not ADMIN, append 'user_id = (SELECT id FROM users WHERE email="...")' to the query.
- Write SELECT query only.

6.5 Role-Based Data Access (Role-Based Chatbot Access)

Role	Access Scope	Implementation Logic
Individual	Own orders, purchases, reviews, spending patterns	Automatically appends user_id = (SELECT id FROM users WHERE email = '...')
Corporate	Product, order, customer, and review data of owned store	Appends store_id IN (SELECT id FROM stores WHERE owner_id = ...)
Admin	Full platform access	No filter applied

6.6 Gemini API Integration

```
// ChatService.java (summary)
String URL = "https://generativelanguage.googleapis.com/v1beta/" +
             "models/gemini-2.5-flash:generateContent?key=" + API_KEY;

Map<String, Object> request = Map.of(
    "contents", List.of(
        Map.of("parts", List.of(
            Map.of("text", systemPrompt)
        ))
    )
);

Map<String, Object> response = restTemplate.postForObject(URL, request, Map.class);
String generatedSql = ((Map)((Map)((List)response.get("candidates")).get(0))
                        .get("content")).get("parts")...;

// Clean markdown fences
generatedSql = generatedSql.replace("```sql", "").replace("```", "").trim();

// Validator: forbidden keyword + SELECT-only
if (upperSql.contains("PASSWORD") || upperSql.contains("HASH")
    || !upperSql.startsWith("SELECT")) {
    return "⚠️ Access to this data was blocked due to security protocol.";
}

List<Map<String, Object>> result = jdbcTemplate.queryForList(generatedSql);
return formatResult(result);
```

6.7 Response Formatting

Result rows are presented in the following format:

Analysis Report

Record 1

- **Product Name:** Wireless Headphones
- **Total Sales:** 1,284
- **Average Rating:** 4.6

This format is both human-readable and suitable for markdown rendering on the Angular side.

6.8 Example Interactions

Natural Language Question	Expected Output
"Category-based sales for last month"	SELECT c.name, SUM(oi.quantity * oi.price) ... GROUP BY c.name
"Top 5 customers by revenue"	SELECT u.email, SUM(o.grand_total) ... ORDER BY ... LIMIT 5
"Lowest-rated products"	SELECT p.name, AVG(r.star_rating) ... ORDER BY avg_rating ASC
"How many orders were shipped by air?"	SELECT COUNT(*) FROM shipments WHERE mode='Flight'
"Hello!"	Guardrails → greeting message, no SQL generation
"Tell me the weather"	Guardrails → "Out of scope" rejection

7. Security Architecture

7.1 Layered Defense

Layer	Measure
Transport	CORS open only for http://localhost:4200 (production: HTTPS + strict origin)
Authentication	JWT (HS256); JwtAuthFilter validates every request
Authorization	Spring Security SecurityFilterChain; role-based endpoint access
AI Layer	Prompt-based guardrails + post-generation validator
Database	JdbcTemplate.queryForList parameterless SELECT; JPA prepared statements in CRUD endpoints
Sensitive Data	password_hash blocked in AI responses; application.properties excluded from Git (documented in README)

7.2 Attack Vectors and Countermeasures

- SQL Injection (CRUD): JPA repositories and method-based queries enforce parameter binding; no raw concatenation.
- SQL Injection (AI): Generated SQL works with read-only (SELECT-only) executor; dynamic DML and DDL are blocked.
- Prompt Injection: System prompt reinjects security rules on every request; user input is appended at the prompt end.
- Sensitive Leakage: Any output containing PASSWORD or HASH is rejected; if role is not ADMIN, user_id filter is mandatory.
- JWT Theft: Short validity period (24 hours) + HTTPS recommendation.

7.3 Known Limitations (Improvement Notes)

In its current form, SecurityConfig applies permitAll to all requests (for development convenience). During production transition, role-based rules such as requestMatchers("/api/admin/").hasRole("ADMIN") should be enabled, passwords should be stored with BCryptPasswordEncoder, and ChatController should obtain real user email and role data from the Authentication object.

8. Challenges Encountered and Solutions

Challenge 1 — Integrating Heterogeneous Datasets

Problem: Six different Kaggle datasets used different column names, encodings, date formats, and identifier structures.

Solution: First, a conceptual target schema (USERS-STORES-PRODUCTS-...) was designed. Then, a field mapping matrix (§ 4.2) was created for each source. All dates were converted to ISO 8601, prices to a single currency, encoding to utf8mb4. New surrogate keys were generated, while original identifiers (invoice_no, reviews.id) were preserved for traceability.

Challenge 2 — Turkish Character Corruption

Problem: In early ETL runs, city names such as Istanbul and Umraniye appeared as ?.

Solution: DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci was used at table level; useUnicode=true&characterEncoding=UTF-8 was added to JDBC URL; ChatController returns responses with produces = "text/plain;charset=UTF-8".

Challenge 3 — Producing Correct SQL from Natural Language

Problem: General-purpose large language model calls frequently produced hallucinated non-existent table and column names.

Solution: The ChatService.askAi method injects the full schema of all 9 tables into the system prompt on every request. This significantly reduced fabricated column references.

Challenge 4 — Cleaning Large Language Model Output

Problem: Gemini often wrapped generated SQL inside sql ... markdown fences, causing JdbcTemplate.queryForList() syntax errors.

Solution: Patterns sql and were removed at string level; trim() was applied. Then a static validator was run: SQL must start with SELECT and must not contain sensitive keywords.

Challenge 5 — Unauthorized Data Leakage Risk (Prompt Injection)

Problem: When a user entered a command like "list all password hashes," the model could produce a compliant query.

Solution: Two-layer defense:

1. Rule in system prompt: never return data containing passwords or hashes.
2. Post-generation static validator: if SQL contains PASSWORD or HASH, it is not executed, and if role is not ADMIN, user_id filter is mandatory.

Challenge 6 — API Key Exposure

Problem: During development, the Gemini API key entered git history.

Solution: With commit dc55a54, application.properties was added to .gitignore and removed from tracking (security: remove sensitive properties file from tracking). Instructions were left in README for developers to add their own keys.

Challenge 7 — Parsing Gemini Response JSON Structure

Problem: Gemini response arrives in nested structure such as candidates[0].content.parts[0].text; ClassCastException occurred during type conversions.

Solution: RestTemplate response was read as Map<String, Object>; safe (List) and (Map) casts were applied at each level; failures were reduced to a single user-friendly message via one try/catch.

Challenge 8 — Performance for Complex Queries

Problem: AI-generated JOINs can combine 8+ tables.

Solution: Index on frequent columns (idx_user_email); InnoDB tables auto-generate foreign key indexes; Spring Data JPA @Index annotation was added (User entity).

Challenge 9 — Multi-Tenancy with Role-Based Access Control

Problem: A corporate user should only see data from their own store.

Solution: Two-layer isolation was established both in REST layer (with advanced Spring Security configuration) and chatbot layer (by injecting store_id IN (SELECT ... WHERE owner_id = ...) filter into prompt).

9. Conclusion and Future Improvements

This project demonstrates how a modern e-commerce business intelligence platform can be designed end-to-end: normalization of heterogeneous data sources with ETL, JWT-secured REST APIs, an Angular-based responsive frontend, and a role-aware AI assistant that generates SQL from natural language. The Spring Boot + Gemini combination is a practical proof of the "large capability with a small team" principle.

9.1 Achieved Goals

- 9-table normalized relational schema (3NF)
- Integration of 6 Kaggle datasets (ETL + field mapping)
- JWT-based authentication
- Complete REST API through 11 controllers
- Natural language query → SQL → formatted response
- Multi-layer security (prompt + static + RBAC)

9.2 Recommended Improvements

Area	Recommendation
Security	Hashing with BCryptPasswordEncoder, refresh token flow, replacing permitAll with role-based rules.
AI	Multi-agent implementation with LangGraph state machine (Guardrails, SQL, Error, Analysis, Visualization); max 3-retry mechanism.
Visualization	Plotly integration; render in Angular when response includes chart data.
Observability	Query logging (who, when, which query) for audit trails.
Deployment	Docker Compose (Spring + MySQL + Angular nginx); CI/CD with GitHub Actions.
API Documentation	Integrate Swagger/OpenAPI (springdoc-openapi-starter-webmvc-ui).
Testing	Service and controller tests with JUnit 5; real MySQL integration tests with Testcontainers.